

Informed search algorithms

Chapter 4

Outline

- Best-first search
- Greedy best-first search
- A^* search
- Heuristics
- Local search algorithms
 - Hill-climbing search
 - Simulated annealing search
 - Local beam search
 - Genetic algorithms

Review: Tree search

```
function TREE-SEARCH(problem, fringe) returns a solution, or failure
  fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
  loop do
    if fringe is empty then return failure
    node ← REMOVE-FRONT(fringe)
    if GOAL-TEST[problem] applied to STATE(node) succeeds return node
    fringe ← INSERTALL(EXPAND(node, problem), fringe)
```

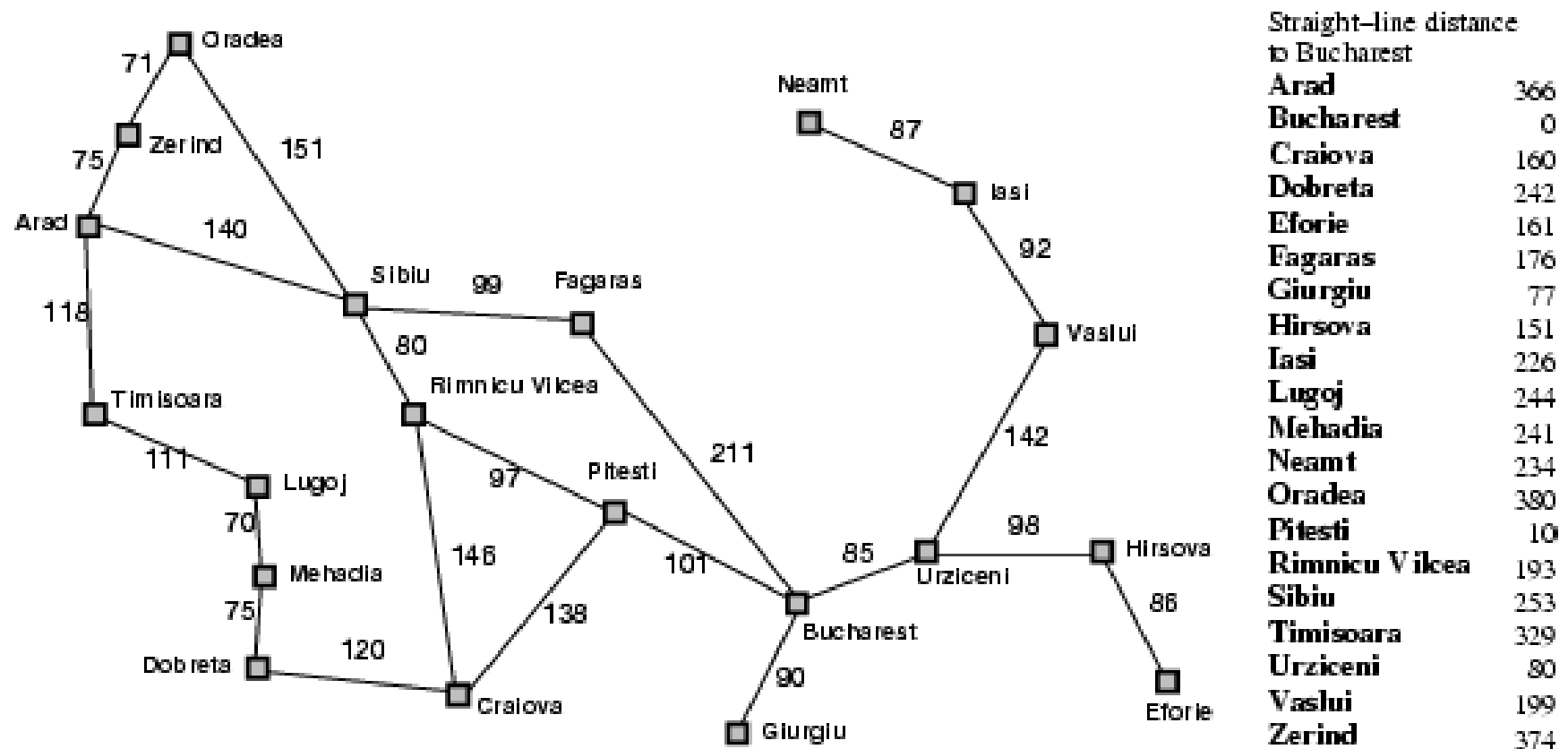
A strategy is defined by picking the **order of node expansion**

Best-first search

- Idea: use an **evaluation function** $f(n)$ for each node
 - estimate of "desirability"
 -
 - Expand most desirable unexpanded node
 -
- Implementation:

Order the nodes in fringe in decreasing order of desirability
- Special cases:
 - greedy best-first search
 - A* search
 -

Romania with step costs in km



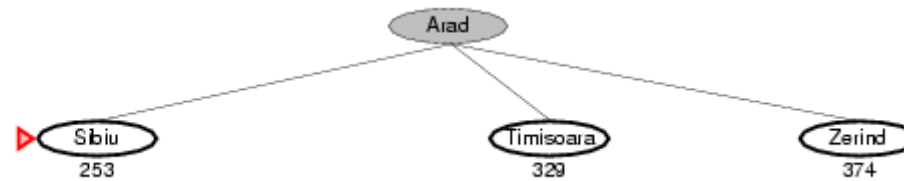
Greedy best-first search

- Evaluation function $f(n) = h(n)$ (**h**euristic)
= estimate of cost from n to *goal*
- e.g., $h_{SLD}(n)$ = straight-line distance from n to Bucharest
-
- Greedy best-first search expands the node that **appears** to be closest to goal
-

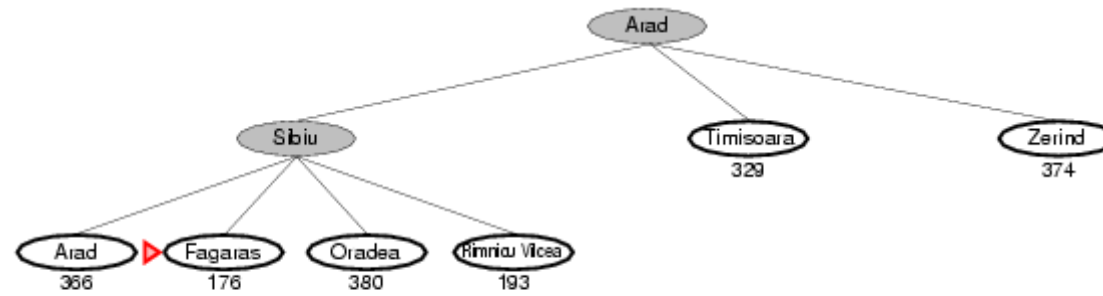
Greedy best-first search example



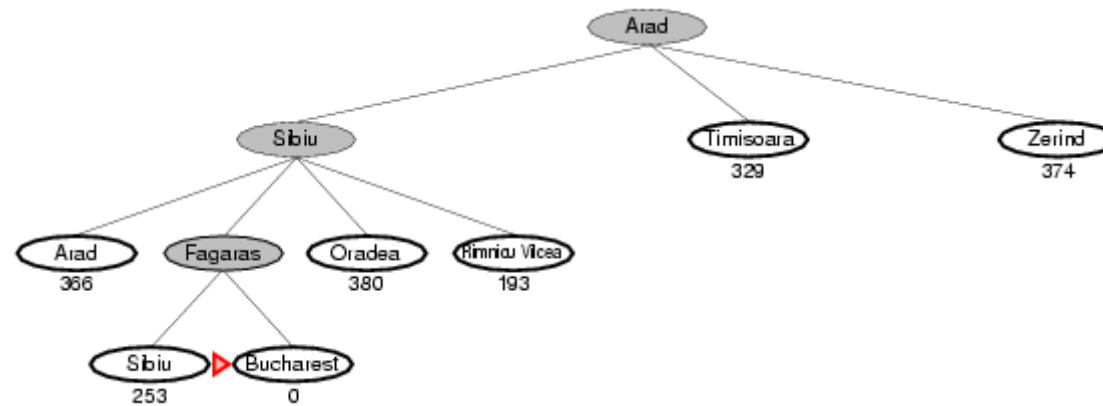
Greedy best-first search example



Greedy best-first search example



Greedy best-first search example



Properties of greedy best-first search

- Complete? No – can get stuck in loops, e.g., lasi $\rightarrow$ Neamt $\rightarrow$ lasi $\rightarrow$ Neamt $\rightarrow$...

Complete in finite space with repeated-state checking

Time? $O(b^m)$, but a good heuristic can give dramatic improvement

Space? $O(b^m)$ -- keeps all nodes in memory

Optimal? No

A* search

- Idea: avoid expanding paths that are already expensive

-

- Evaluation function $f(n) = g(n) + h(n)$

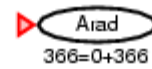
-

$g(n)$ = cost so far to reach n

$h(n)$ = estimated cost from n to goal

$f(n)$ = estimated total cost of path through n to goal

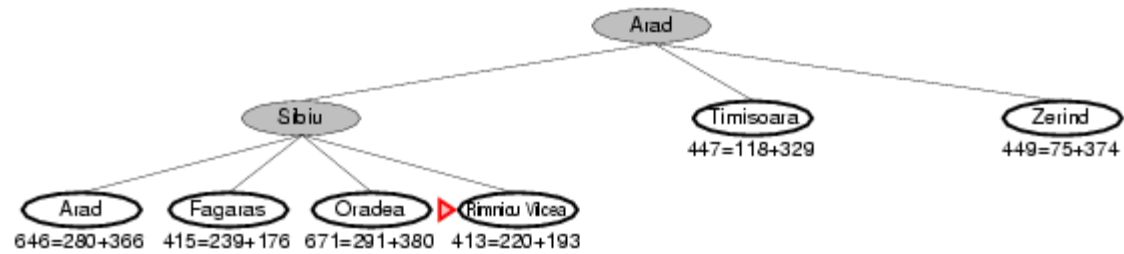
A* search example



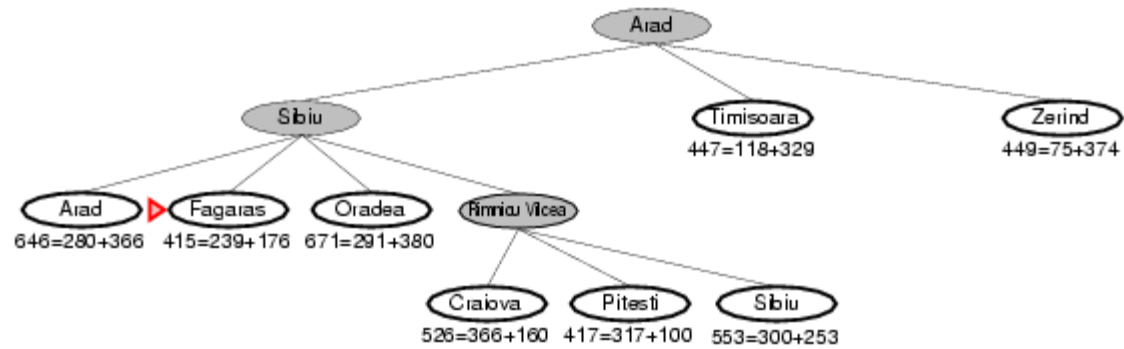
A* search example



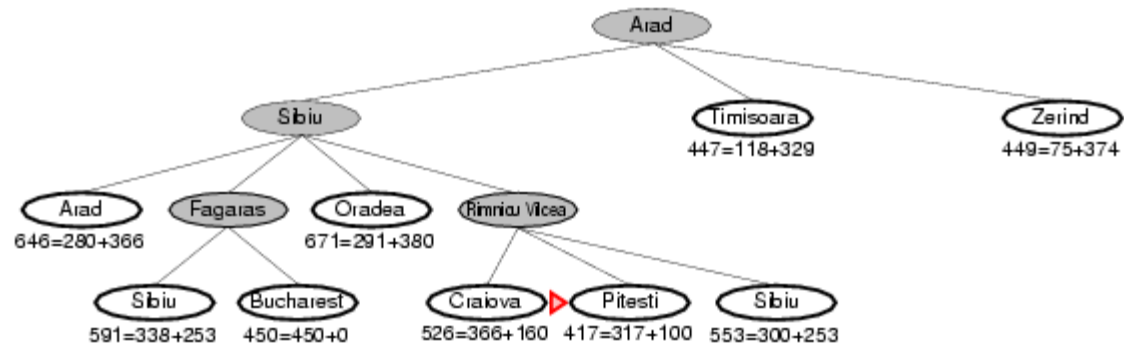
A* search example



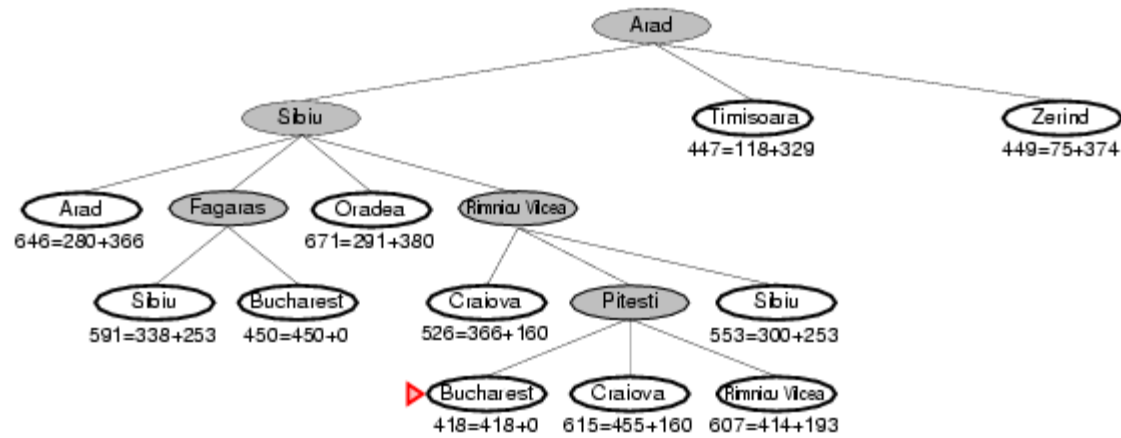
A* search example



A* search example



A* search example

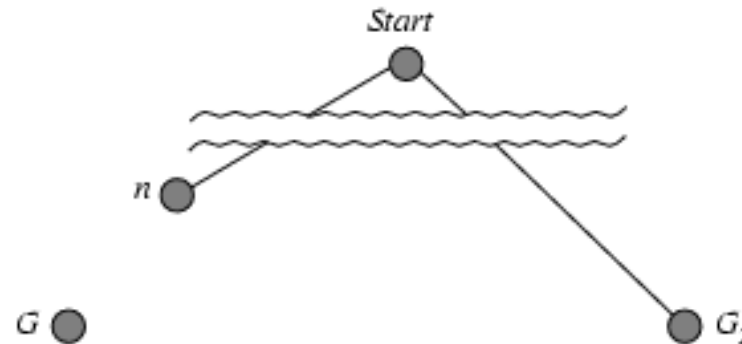


Admissible heuristics

- A heuristic $h(n)$ is **admissible** if for every node n , $0 \leq h(n) \leq h^*(n)$, where $h^*(n)$ is the **true** cost to reach the goal state from n .
- An admissible heuristic **never overestimates** the cost to reach the goal, i.e., it is **optimistic**
- Example: $h_{SLD}(n)$ (never overestimates the actual road distance)
- **Theorem:** If $h(n)$ is admissible, A^* using TREE-SEARCH is optimal
-

Optimality of A^* (proof)

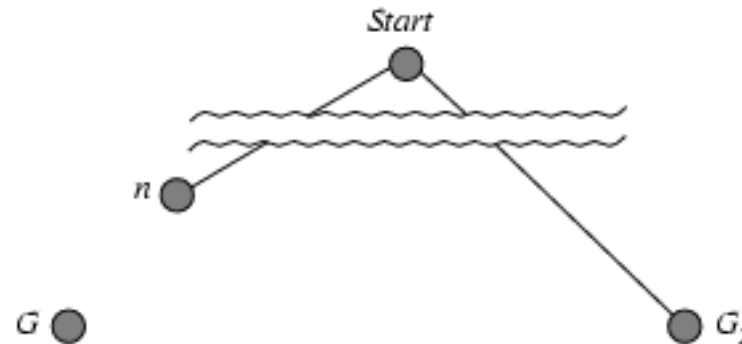
- Suppose some suboptimal goal G_2 has been generated and is in the fringe. Let n be an unexpanded node in the fringe such that n is on a shortest path to an optimal goal G .



- $f(G_2) = g(G_2)$ since $h(G_2) = 0$
- $g(G_2) > g(G)$ since G_2 is suboptimal
- $f(G) = g(G)$ since $h(G) = 0$
- $f(G_2) > f(G)$ from above

Optimality of A^* (proof)

- Suppose some suboptimal goal G_2 has been generated and is in the fringe. Let n be an unexpanded node in the fringe such that n is on a shortest path to an optimal goal G .



- $f(G_2) > f(G)$ from above
- $h(n) \leq h^*(n)$ since h is admissible
- $g(n) + h(n) \leq g(n) + h^*(n)$
- $f(n) \leq f(G)$
- Hence $f(G_2) > f(n)$, and A^* will never select G_2 for expansion

- **A***的最有效是源于已知路径+未知路径估算的总体优化（比较 **min-cost**, **greedy**）
- **A***对于树搜索是最优的，对于图搜索则未必
 - 为了使算法最优，可以扩展图搜索算法使它丢弃重复路径中**cost**大的路径，或
 - 保证到达任何重复状态的最优路径总是第一条被选择：一致性启发函数。

Consistent heuristics

- A heuristic is **consistent** if for every node n , every successor n' of n generated by any action a ,

-

$$h(n) \leq c(n,a,n') + h(n')$$

- If h is consistent, we have

-

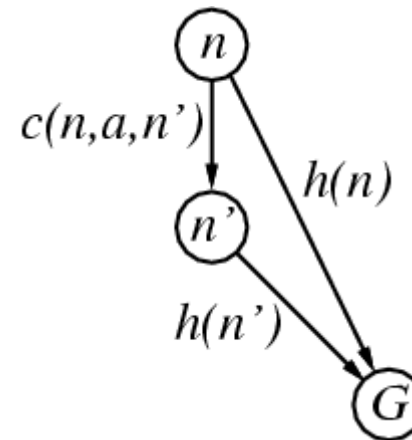
$$\begin{aligned} f(n') &= g(n') + h(n') \\ &= (g(n) + c(n,a,n')) + h(n') \\ &\geq g(n) + h(n) \\ &= f(n) \end{aligned}$$

- i.e., $f(n)$ is non-decreasing along any path.

-

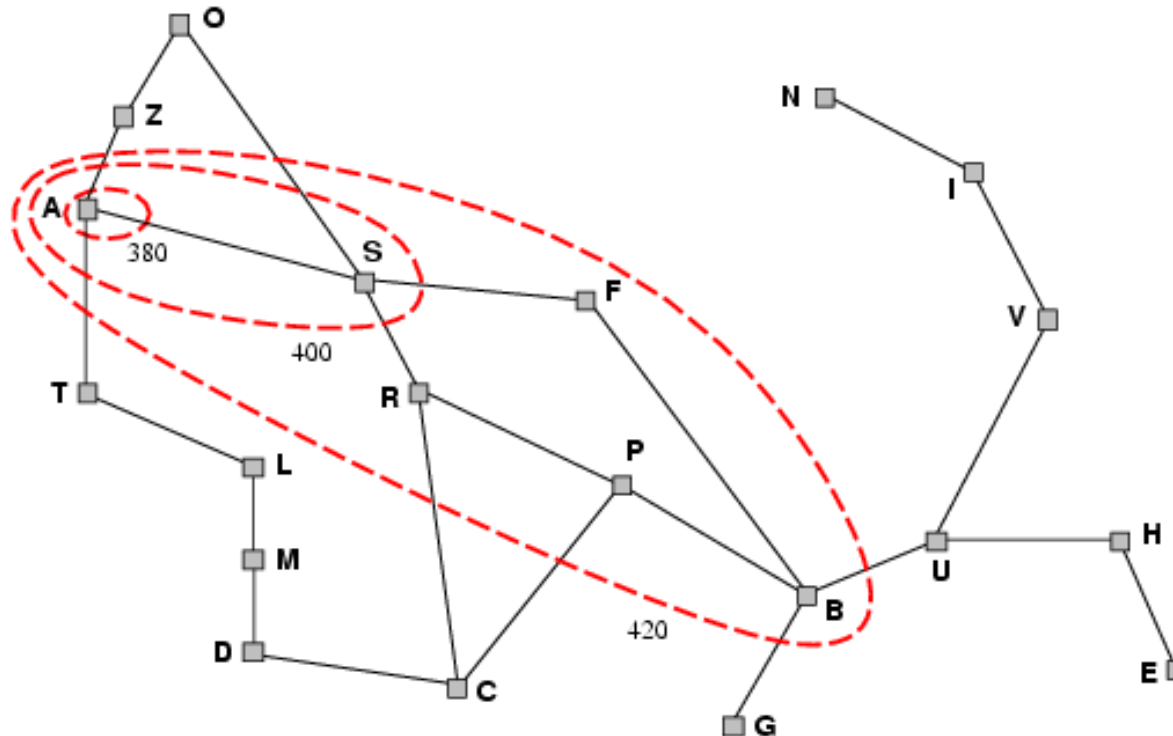
- **Theorem:** If $h(n)$ is consistent, A* using GRAPH-SEARCH is optimal

-



Optimality of A^*

- A^* expands nodes in order of increasing f value
-
- Gradually adds " f -contours" of nodes
- Contour i has all nodes with $f=f_i$, where $f_i < f_{i+1}$
-



Properties of A*

- Complete? Yes (unless there are infinitely many nodes with $f \leq f(G)$)
- Time? Exponential
- Space? Keeps all nodes in memory
- Optimal? Yes
- - A* expands all nodes with $f(n) < C^*$
 - A* expands some nodes with $f(n) = C^*$
 - A* expands no nodes with $f(n) > C^*$

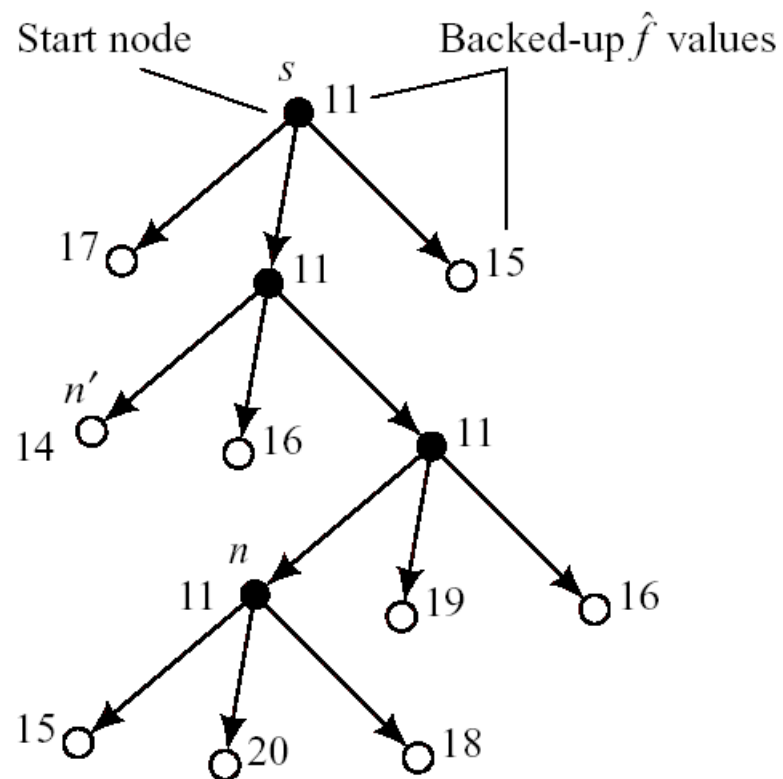
- 图搜索是完备的、最优的，但是对于大多数问题来说，处于目标等值线搜索空间内的节点数是解长度的指数函数（它在内存空间中保存了所有的节点），所以内存不够成了大问题。
- 可以限制使用的存储。如类似IDS，我们可得到IDA*，只是cutoff值是f-cost，而每次新的cutoff值是超过上次迭代的最小的f-cost (The lowest f-cost values of the nodes visited (not expanded) in the previous search is used as the new cut-off value in the next search)
- 改进算法： RBFS和MA* 等。

Recursive Best First Search (RBFS)

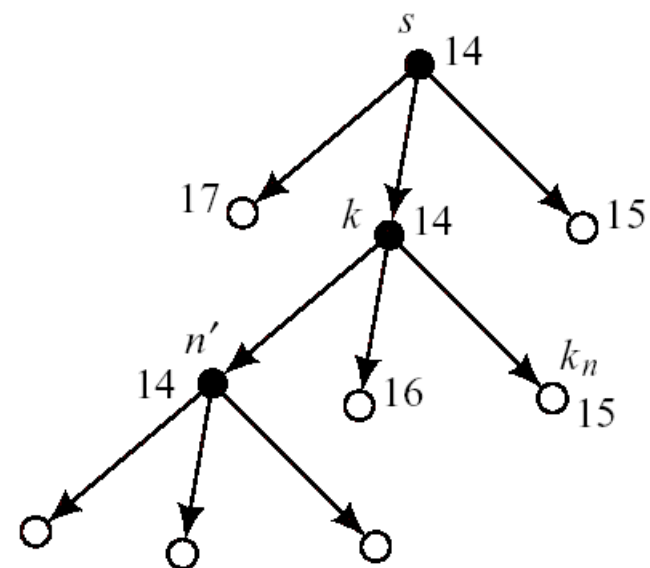
- **Idea:** Try something like depth first search, but let's not forget everything about the branches we have partially explored.
- *We remember the best $f(n)$ value we have found so far in the branch we are abandoning.*
- Backing up f-cost value
 - When a node n is expanded, computes f-cost values of its successors and recomputes f-cost values of n and all of n 's ancestors
- Process of backing up
 - Backed-up f-cost value of node n is the minimum f-cost value of its successors

Recursive Best-First Search

- Description
 - One of successors of node n has the smallest f-cost over all **fringe** nodes, it is expanded in turn, and so on.
 - When another **fringe** node, n' , (not a successor of n) has the lowest value of f-cost
 - backtracks to the lowest common ancestor, node k
 - k_n : successor of node k on the path to n
 - removes the subtree rooted at k_n , from **fringe**
 - k_n becomes an **fringe** node with f-cost value equal to its backed-up value
 - Search continues with that **fringe** node with the lowest f-cost value



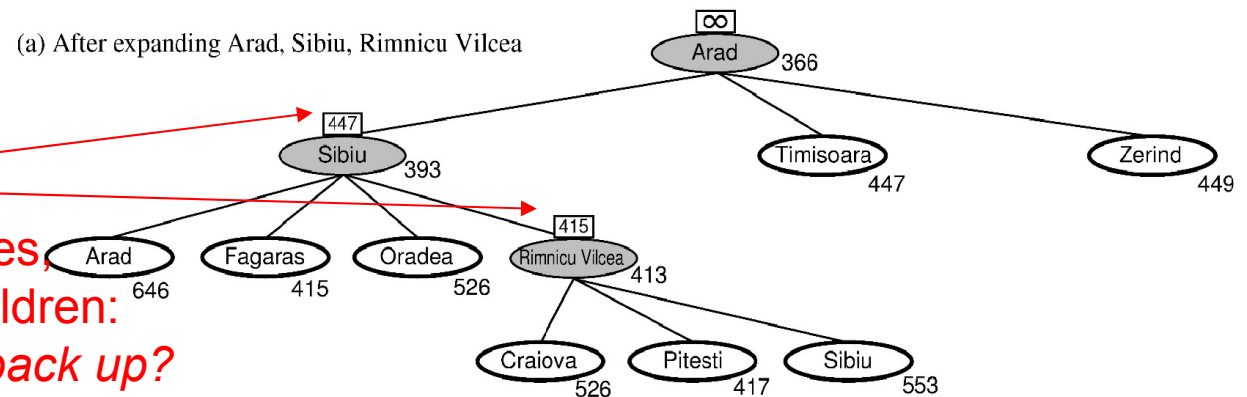
(a) RBFS has just expanded node n but has not yet backed up the $\hat{f}$ values of its successors



(b) $\hat{f}$ values have been backed up, the subtree below k_n has been discarded, and search continues below n'

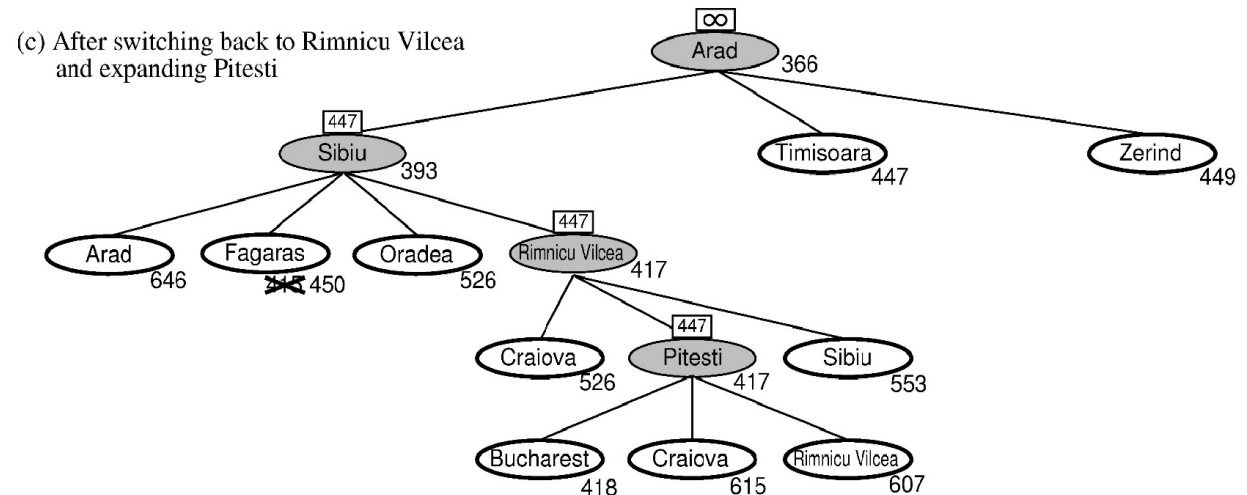
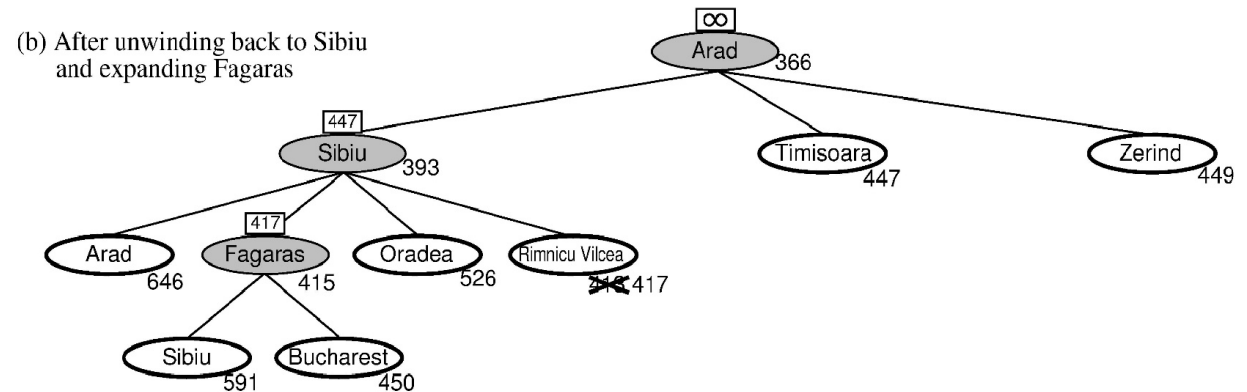
Recursive Best-First Search Example

best alternative
over frontier nodes,
which are not children:
i.e. do I want to back up?



RBFS changes its mind
very often in practice.

This is because the $f=g+h$ become more accurate (less optimistic) as we approach the goal. Hence, higher level nodes have smaller f -values and will be explored first.



Problem: We should keep
in memory whatever we can.

Simple Memory-bounded A^* (SMA *)

- This is like A^* , but when memory is full we delete the worst node (largest f-value).
- Like RBFS, SMA * backs up the value of the forgotten node to its parent.
- If there is a tie (equal f-values) we delete the oldest nodes first.
- SMA * finds the optimal *reachable* solution given the memory constraint.
- Time can still be exponential.

A solution is not reachable
if a single path from root to goal
does not fit into memory

Admissible heuristics

E.g., for the 8-puzzle:

- $h_1(n)$ = number of misplaced tiles
- $h_2(n)$ = total Manhattan distance
(i.e., no. of squares from desired location of each tile)

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

- $h_1(S) = ?$
- $h_2(S) = ?$
-

Admissible heuristics

E.g., for the 8-puzzle:

- $h_1(n)$ = number of misplaced tiles
- $h_2(n)$ = total Manhattan distance
(i.e., no. of squares from desired location of each tile)

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

- $h_1(S) = ?$ 8
- $h_2(S) = ?$ $3+1+2+2+2+3+3+2 = 18$

Dominance

- If $h_2(n) \geq h_1(n)$ for all n (both admissible)
- then h_2 **dominates** h_1
- h_2 is better for search
-
- Typical search costs (average number of nodes expanded):
-
- $d=12$ IDS = 3,644,035 nodes
 $A^*(h_1) = 227$ nodes
 $A^*(h_2) = 73$ nodes
- $d=24$ IDS = too many nodes
 $A^*(h_1) = 39,135$ nodes
 $A^*(h_2) = 1,641$ nodes

Relaxed problems

- A problem with fewer restrictions on the actions is called a **relaxed problem**
- The cost of an optimal solution to a relaxed problem is an admissible heuristic for the original problem
- If the rules of the 8-puzzle are relaxed so that a tile can move **anywhere**, then $h_1(n)$ gives the shortest solution
-
- If the rules are relaxed so that a tile can move to **any adjacent square**, then $h_2(n)$ gives the shortest solution
-

Key point: the optimal solution cost of a relaxed problem is no greater than the optimal solution cost of the real problem

subproblem

- Admissible heuristics can also be derived from the solution cost of a subproblem of a given problem. The cost of the optimal solution of this subproblem is a lower bound on the cost of the complete problem. It turns out to be substantially more accurate than Manhattan distance in some cases.

*	2	4
*		*
*	3	1

Start State

	1	2
3	4	*
*	*	*

Goal State

- The idea behind **pattern databases** is to store these exact solution costs for every possible subproblem instance

Summary

Heuristic functions estimate costs of shortest paths

Good heuristics can dramatically reduce search cost

Greedy best-first search expands lowest h

- incomplete and not always optimal

A* search expands lowest $g + h$

- complete and optimal
- also optimally efficient (up to tie-breaks, for forward search)

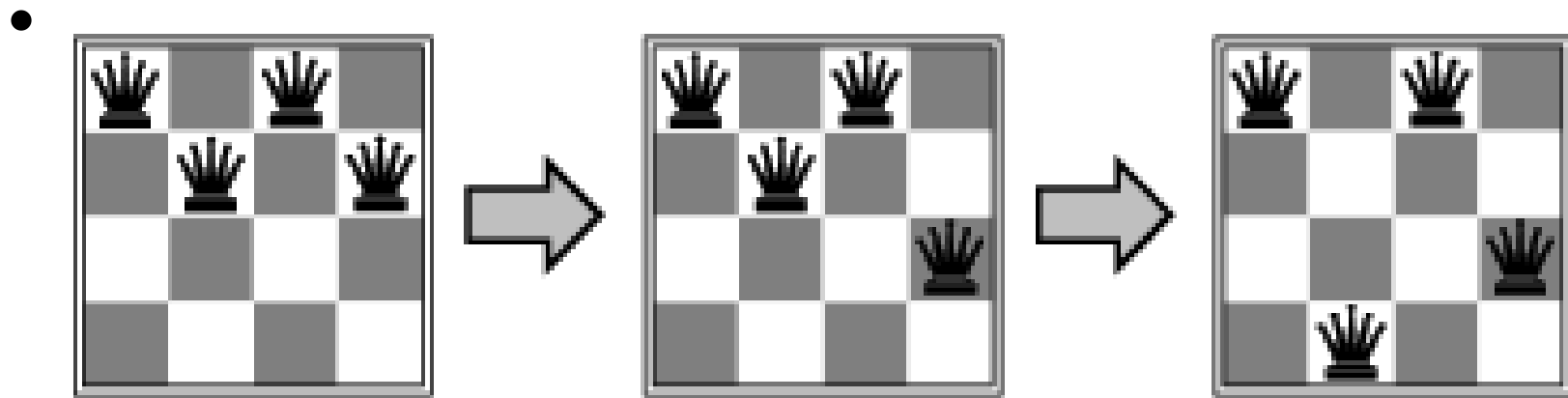
Admissible heuristics can be derived from exact solution of relaxed problems

Local search algorithms

- In many optimization problems, the **path** to the goal is irrelevant; the goal state itself is the solution
-
- State space = set of "complete" configurations
- Find configuration satisfying constraints, e.g., n-queens
- In such cases, we can use **local search algorithms**
 - keep a single "current" state, try to improve it

Example: n -queens

- Put n queens on an $n \times n$ board with no two queens on the same row, column, or diagonal



Move a queen to reduce number of conflicts

局部搜索算法

- 爬山法搜索
- 模拟退火搜索
- 局部剪枝搜索
- 遗传算法

Hill-climbing search

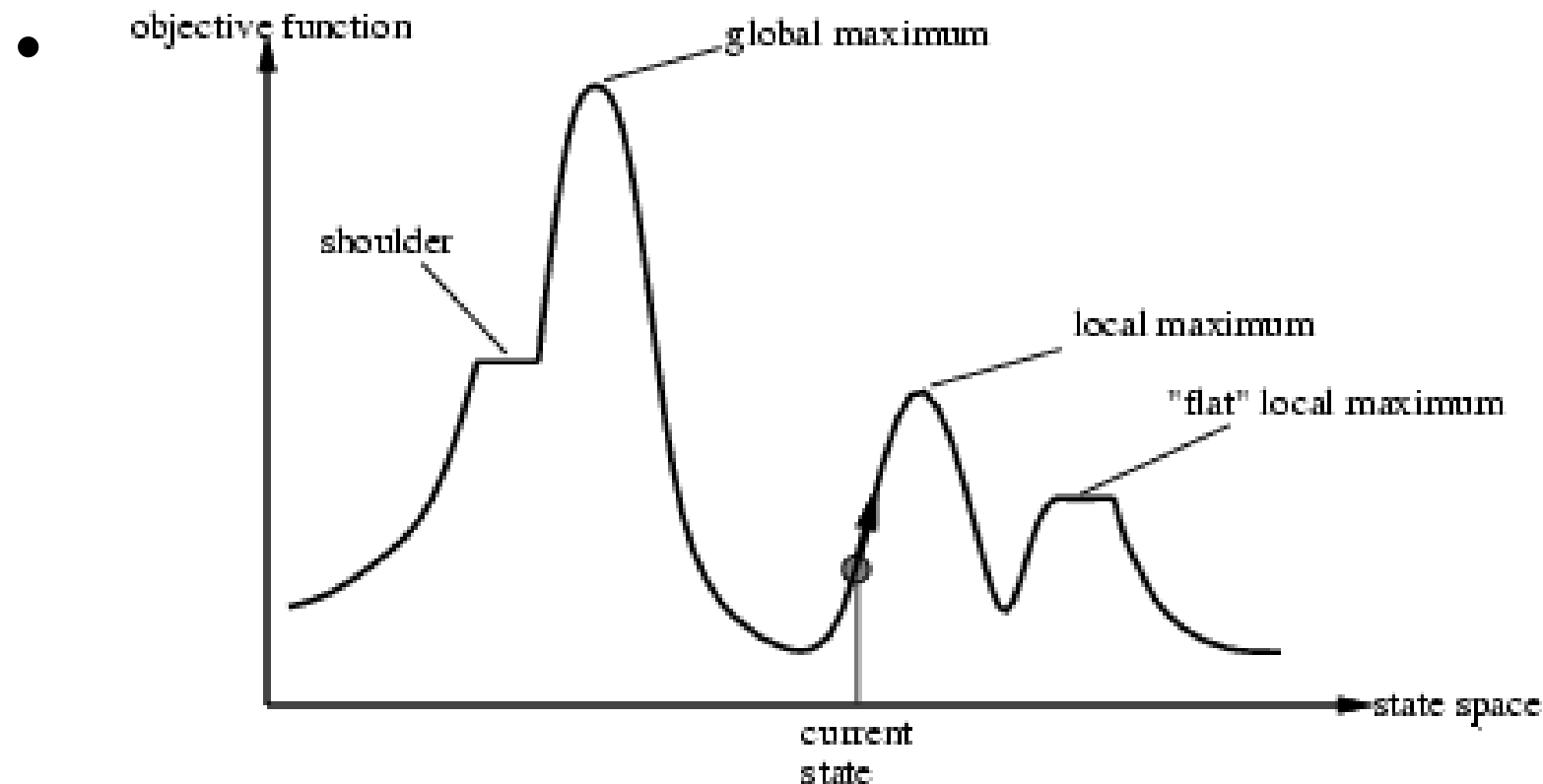
- "Like climbing Everest in thick fog with amnesia"

- ```
function HILL-CLIMBING(problem) returns a state that is a local maximum
 inputs: problem, a problem
 local variables: current, a node
 neighbor, a node

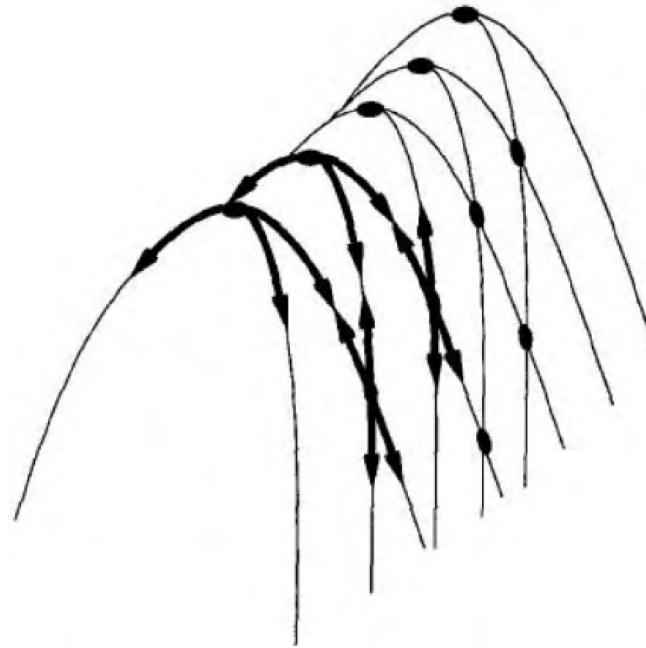
 current ← MAKE-NODE(INITIAL-STATE[problem])
 loop do
 neighbor ← a highest-valued successor of current
 if VALUE[neighbor] ≤ VALUE[current] then return STATE[current]
 current ← neighbor
```

# Hill-climbing search

- Problem: depending on initial state, can get stuck in local maxima



# ridges



---

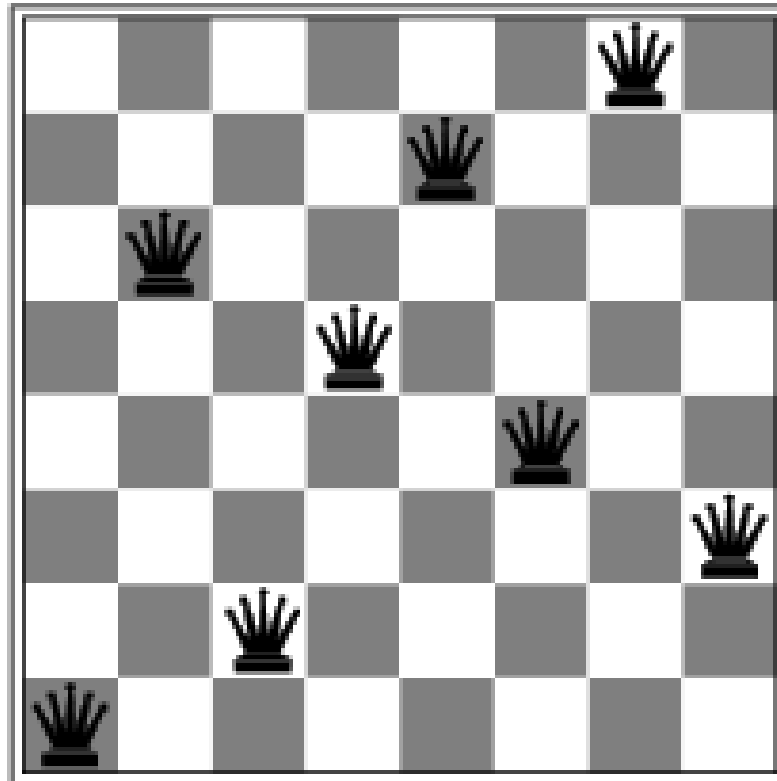
**Figure 4.13** Illustration of why ridges cause difficulties for hill-climbing. The grid of states (dark circles) is superimposed on a ridge rising from left to right, creating a sequence of local maxima that are not directly connected to each other. From each local maximum, all the available actions point downhill.

# Hill-climbing search: 8-queens problem

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 18 | 12 | 14 | 13 | 13 | 12 | 14 | 14 |
| 14 | 16 | 13 | 15 | 12 | 14 | 12 | 16 |
| 14 | 12 | 18 | 13 | 15 | 12 | 14 | 14 |
| 15 | 14 | 14 | ♚  | 13 | 16 | 13 | 16 |
| ♚  | 14 | 17 | 15 | ♚  | 14 | 16 | 16 |
| 17 | ♚  | 16 | 18 | 15 | ♚  | 15 | ♚  |
| 18 | 14 | ♚  | 15 | 15 | 14 | ♚  | 16 |
| 14 | 14 | 13 | 17 | 12 | 14 | 12 | 18 |

- $h$  = number of pairs of queens that are attacking each other, either directly or indirectly
- $h = 17$  for the above state
-

# Hill-climbing search: 8-queens problem



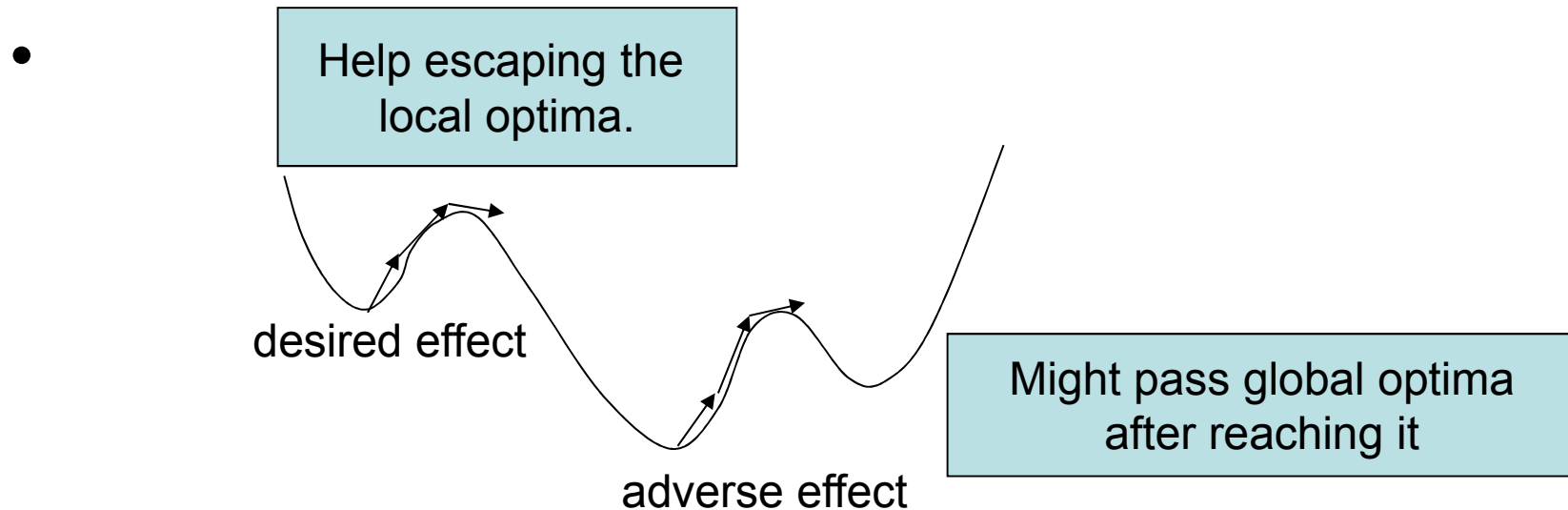
- A local minimum with  $h = 1$
-

# 避免局部极值

- 随机重新开始
  - 完备的概率接近1
  - 但效率低下

# Simulated annealing search

- 退火：为了增强金属和玻璃的韧性和硬度而先把它们加热到高温然后逐渐冷却的过程
- Idea: escape local maxima by allowing some “bad” moves but **gradually decrease** their frequency





# Simulated annealing search

```
function SIMULATED-ANNEALING(problem, schedule) returns a solution state
 inputs: problem, a problem
 schedule, a mapping from time to “temperature”
 local variables: current, a node
 next, a node
 T, a “temperature” controlling prob. of downward steps

 current ← MAKE-NODE(INITIAL-STATE[problem])
 for t ← 1 to ∞ do
 T ← schedule[t]
 if T = 0 then return current
 next ← a randomly selected successor of current
 $\Delta E \leftarrow \text{VALUE}[\textit{next}] - \text{VALUE}[\textit{current}]$
 if $\Delta E > 0$ then current ← next
 else current ← next only with probability $e^{\Delta E/T}$
```

# simulated annealing search

- 以某个概率改变后继： 需要用到(0,1)之间随机数
- 温度下降策略：
  - $T[1]=Schedule[1]$  使得  $\exp(-\Delta/T[1])) \approx 1$
  - $T[k+1]= \alpha \bullet T[k]$ , 其中 $\alpha$  为小于但接近1的常数.
- One can prove: If  $T$  decreases slowly enough, then simulated annealing search will find a global optimum with probability approaching 1
- 
- Widely used in VLSI layout, airline scheduling, etc

# Local beam search

- Keep track of  $k$  states rather than just one
- 
- Start with  $k$  randomly generated states
- 
- At each iteration, all the successors of all  $k$  states are generated
- 
- If any one is a goal state, stop; else select the  $k$  best successors from the complete list and repeat.

# Stochastic beam search

- $k$  states focus on a small region of state space
  - 缺乏多样化
  -
- Stochastic beams search: choose  $k$  successors randomly with good successor having high probability
- 自然选择：强者多生

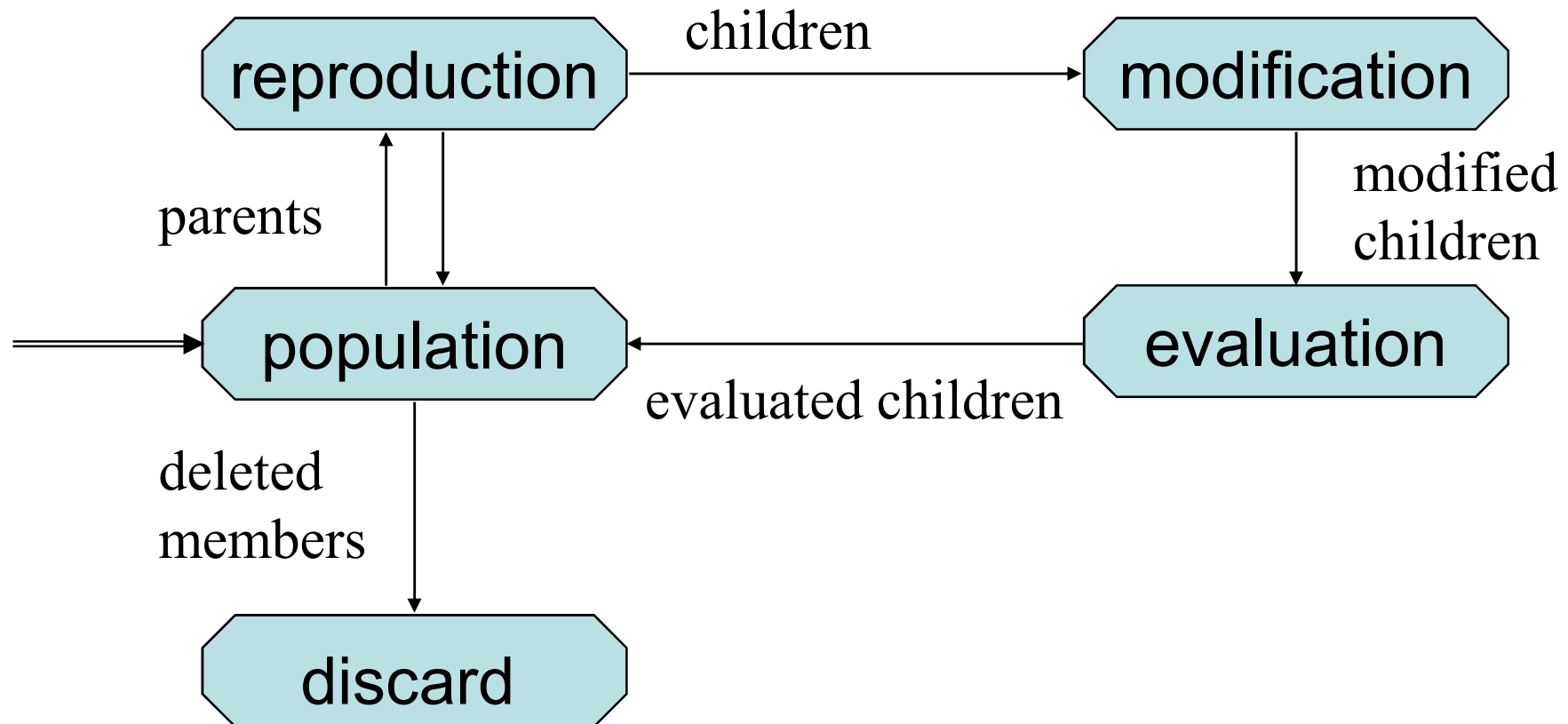
# Genetic algorithms

- A successor state is generated by combining two parent states
- 
- Start with  $k$  randomly generated states (**population**)
- 
- A state is represented as a string over a finite alphabet (often a string of 0s and 1s)
- 
- Evaluation function (**fitness function**). Higher values for better states.
- 
- Produce the next generation of states by selection, crossover, and mutation
-

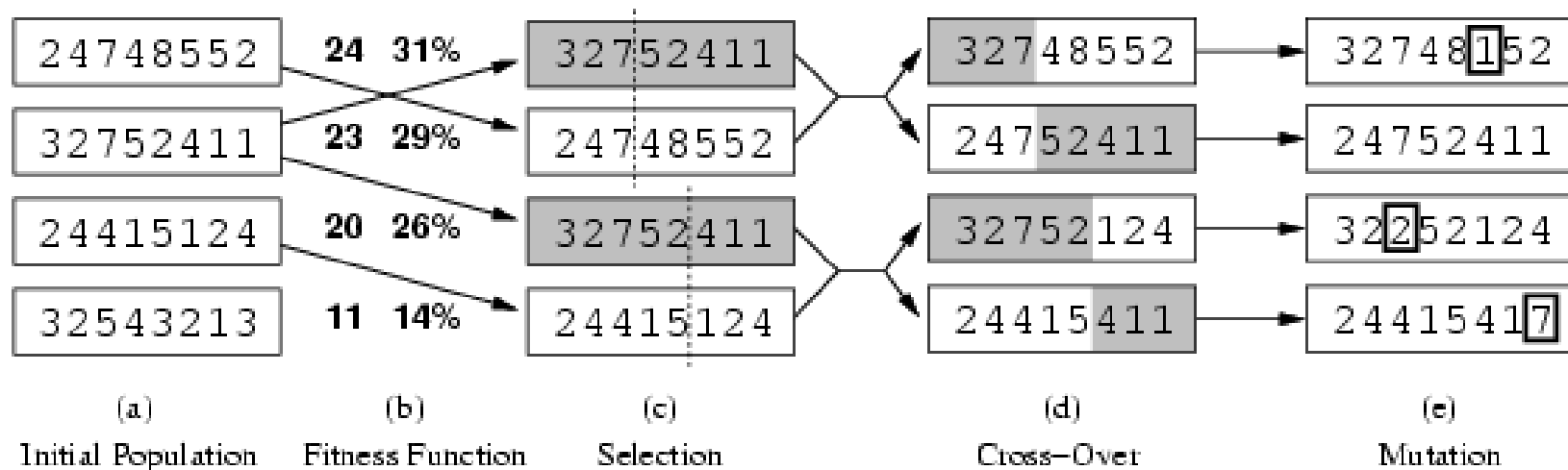
# Simple Genetic Algorithm

```
{
 initialize population;
 evaluate population;
 while TerminationCriteriaNotSatisfied
 {
 select parents for reproduction;
 perform crossover and mutation;
 evaluate population;
 }
}
```

# The GA Cycle



# Genetic algorithms

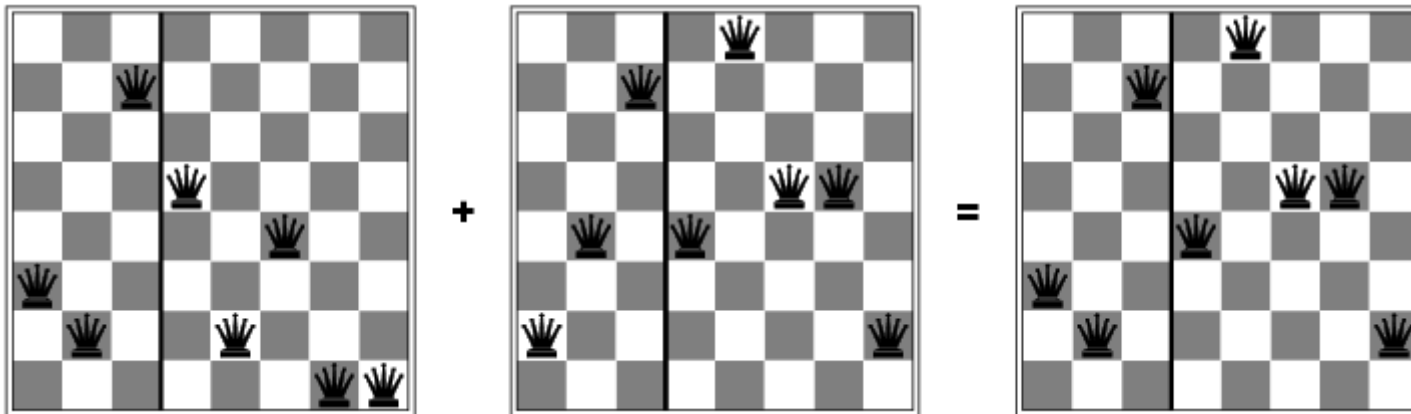


- Fitness function: number of non-attacking pairs of queens (min = 0, max =  $8 \times 7/2 = 28$ )
- 
- $24/(24+23+20+11) = 31\%$
- 
- $23/(24+23+20+11) = 29\%$  etc



# Genetic algorithms

Crossover helps **iff** substrings are meaningful components



# Issues of GA

- Choosing basic implementation issues:
  - representation
  - population size, mutation rate, ...
  - selection, deletion policies
  - crossover, mutation operators
- Termination Criteria
- Performance, scalability
- Solution is only as good as the evaluation function (often hardest part)

# 作业

- 阅读书籍：
  - K. A. De Jong, Evolutionary computation: a unified approach
- 4.1 Trace the operation of  $A^*$  search applied to the problem of getting to Bucharest from Lugoj using the straight-line distance heuristic. That is, show the sequence of nodes that the algorithm will consider and the  $f$ ,  $g$ , and  $h$  score for each node

- 4.8 The traveling salesperson problem (TSP) can be solved via the minimum spanning tree (MST) heuristic, which is used to estimate the cost of completing a tour, given that a partial tour has already been constructed. The MST cost of a set of cities is the smallest sum of the link costs of any tree that connects all the cities.
- a. Show how this heuristic can be derived from a relaxed version of the TSP.
- b. Show that the MST heuristic dominates straight-line distance.
- c. Write a problem generator for instances of the TSP where cities are represented by random points in the unit square.
- d. Find an efficient algorithm in the literature for constructing the MST, and use it with an admissible search algorithm to solve instances of the TSP.